Advanced Data Structures

Persistent segment trees, merge-sort trees, implicit treaps

Learning goals

- Make a segment tree **persistent** with path copying; size its memory
- Answer **k-th smallest in a range** in O(log n) with version differences
- Build a **merge-sort tree** and know when offline beats it
- Prove a **treap** has expected O(log n) depth
- Use an **implicit treap** as an array with cut / paste / reverse

When a plain segment tree is not enough

Shape	Example	Tool
positions × **values**	k-th smallest in a[l..r]	persistent / merge-sort tree
old versions	"copy array k", undo	persistent segtree
array **changes shape**	cut & paste, reverse	implicit treap

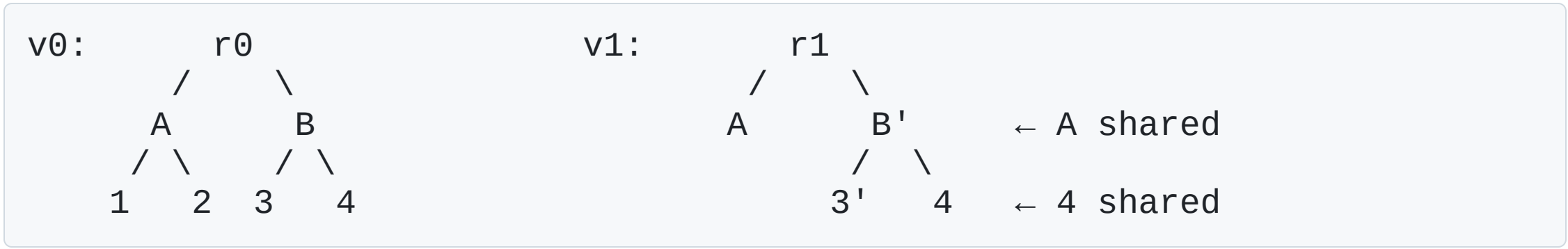
Motivating problem — K-th Number

Static array, n, q ≤ 10^5. Query (l, r, k): k-th smallest of a[l..r].

- Sorting per query: O(q n log n) — too slow
- Mo + order statistics: O(n√q log n) — borderline
- **Persistent segment tree: O((n + q) log n)**

Path copying

A point update touches only $\lceil \log_2 n \rceil + 1$ nodes. **Copy them** instead of modifying.



O(log n) new nodes per update; old roots stay valid forever.

Implementation consequences

- Children are shared → **explicit** `L[v]`, `R[v]` (no `2v, 2v+1`)
- Node 0 = empty tree, `L[0] = R[0] = 0` → no null checks
- Memory: $2n + q \cdot (\lceil \log_2 n \rceil + 1)$ nodes
  - $n = q = 2 \cdot 10^5$ → ~$4.2 \cdot 10^6$ nodes ≈ 70 MB with `ll` sums
- **O(1) copy of an entire array** = copy the root index

Persistent update (code)

```
int update(int prev, int lo, int hi, int pos, ll x) {
    int v = ++cnt;                          // new node
    lc[v] = lc[prev]; rc[v] = rc[prev];     // share children
    if (lo == hi) { sm[v] = x; return v; }
    int mid = (lo + hi) / 2;
    if (pos <= mid) lc[v] = update(lc[prev], lo, mid, pos, x);
    else            rc[v] = update(rc[prev], mid + 1, hi, pos, x);
    sm[v] = sm[lc[v]] + sm[rc[v]];
    return v;
}
// copy array k:   roots.push_back(roots[k]);
```

CSES 1737 *Range Queries and Copies*.

K-th smallest: trees over values

- Compress values to 0..m−1; leaf = count of that value
- `root[i]` = tree of multiset {a[1..i]} (insert a[i] into `root[i−1]`)
- **Counts subtract:**
  `cnt(root[r], [x,y]) − cnt(root[l−1], [x,y])` = #{i ∈ [l,r] : a[i] ∈ [x,y]}

The pair (root[r], root[l−1]) acts like one tree for a[l..r].

Walking two trees at once

```
int kth(int u, int v, int lo, int hi, int k) {   // u = root[r], v = root[l-1]
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        int c = cnt[L[u]] - cnt[L[v]];            // # in left value half
        if (k <= c) { u = L[u]; v = L[v]; hi = mid; }
        else { k -= c; u = R[u]; v = R[v]; lo = mid + 1; }
    }
    return lo;                                    // compressed value
}
```

Invariant: answer = k-th smallest of a[l..r] within [lo, hi]. O(log n).

Same idea, other problems

- **count ≤ x in [l, r]** online: prefix query on version difference
- **distinct values in [l, r]** online (DQUERY):
  at i, +1 at i and −1 at previous occurrence of a[i];
  query version r on positions [l, r]
- **Binary search over versions** (CF 484E *Sign on Fence*):
  insert planks by decreasing height; smallest version with a run ≥ w

Pattern: *sort by one parameter, one version per prefix*.

Merge-sort tree

Segment tree over positions; node stores the **sorted** values of its segment.

```
[1..8]:  1 2 3 4 5 6 7 8
[1..4]:  2 4 5 7     [5..8]:  1 3 6 8
[1..2]:  5 7  ...
```

- Build: `std::merge` children → O(n log n) time & memory
- countLE(l, r, x): O(log n) nodes × `upper_bound` = **O(log² n)**
- k-th: binary search on answer → O(log³ n) (prefer persistent)

Merge-sort tree query (code)

```
int countLE(int v, int lo, int hi, int l, int r, int x) const {
    if (r < lo || hi < l) return 0;
    if (l <= lo && hi <= r)
        return upper_bound(t[v].begin(), t[v].end(), x) - t[v].begin();
    int mid = (lo + hi) / 2;
    return countLE(2 * v, lo, mid, l, r, x)
         + countLE(2 * v + 1, mid + 1, hi, l, r, x);
}
```

Variants: prefix sums per node → *sum of values ≤ x*;

ordered set per node → point updates in $O(\log^2 n)$.

Offline beats both

If all queries are known: sort elements and queries by value, sweep, Fenwick tree over positions.

- O((n + q) log n), O(n) memory, 15 lines
- Intended solution of SPOJ **KQUERY**

> First question to ask: **are the queries online?**

Treap = BST by key + heap by random priority

- For fixed (key, priority) pairs the treap is **unique**
- Root = max priority; recurse on smaller / larger keys

Expected depth. x_i is an ancestor of $x_j \Leftrightarrow x_i$ has the max priority among x_i..x_j ⇒ $P = 1 / (|i-j| + 1)$.

$E[\text{depth}(x_j)] = \Sigma\ 1/(|i-j|+1) < 2H_n \approx 2 \ln n$ — for **any** input order.

Split and merge

- `split(t, k) → (a, b)`: first k elements / the rest
- `merge(a, b)`: concatenate (all of a before all of b)

Operation	Recipe
insert at p	split(p), merge(merge(a, new), b)
erase p	split(p−1), split(1), merge(a, c)
range op [l, r]	split out middle, tag its root, merge back

Each walks one root-to-leaf path: expected O(log n).

Implicit keys

Do not store keys. Key = **position** in in-order = size(left) + 1.

- `split` by count uses only subtree sizes
- Positions stay correct after cut / paste / insert in the middle

→ An **array** with O(log n) insert, delete, cut, paste, reverse.

Split and merge (code)

```
void split(int x, int k, int& a, int& b) {   // a = first k
    if (!x) { a = b = 0; return; }
    push(x);
    if (T[T[x].l].sz < k) { split(T[x].r, k - T[T[x].l].sz - 1, T[x].r, b); a = x; }
    else                  { split(T[x].l, k, a, T[x].l); b = x; }
    pull(x);
}
int merge(int a, int b) {
    if (!a || !b) return a ? a : b;
    if (T[a].pri > T[b].pri) { push(a); T[a].r = merge(T[a].r, b); pull(a); return a; }
    else                     { push(b); T[b].l = merge(a, T[b].l); pull(b); return b; }
}
```

Lazy reverse

```
void push(int x) {
    if (x && T[x].rev) {
        swap(T[x].l, T[x].r);
        if (T[x].l) T[T[x].l].rev ^= 1;
        if (T[x].r) T[T[x].r].rev ^= 1;
        T[x].rev = false;
    }
}
// reverse [l, r]:
split(root, l - 1, a, b); split(b, r - l + 1, b, c);
T[b].rev ^= 1;
root = merge(merge(a, b), c);
```

Asymmetric aggregates (best prefix) must swap prefix/suffix too.

push / pull discipline

- `push(x)` **before** reading or changing x's children
- `pull(x)` **after** changing them
- In split / merge: push at entry, pull at exit
- Printing the sequence: in-order walk **with** push

Bug symptom: passes until the first reverse.

Complexity summary

Structure	Build	Query	Update	Memory
Persistent segtree	O(n log n)	O(log n)	O(log n) new nodes	O(n + q log n)
Merge-sort tree	O(n log n)	O(log² n)	— (static)	O(n log n)
Implicit treap	O(n log n)	O(log n) exp.	O(log n) exp.	O(n)

Which one?

Need	Use
k-th in range, online	persistent segtree
count ≤ x in range, offline	sort + Fenwick
count ≤ x, online, static	merge-sort tree / persistent
+ point updates	segtree/BIT of ordered sets
versions, O(1) copies	persistent segtree
insert / delete / reverse in middle	implicit treap

Common mistakes

- `2v, 2v+1` indexing in a persistent tree
- Under-allocated node pool (count `2n + q(log n + 2)`)
- Lazy push that **mutates a shared node** — copy first
- `root[0]` not the empty tree → off-by-one on l − 1
- Treap: missing push/pull; `rand()` priorities (RAND_MAX = 32767)
- Persistence where an offline sweep suffices

Practice problems

- CSES 1737 Range Queries and Copies
- SPOJ MKTHNUM · SPOJ DQUERY · SPOJ KQUERY
- CF 707D Persistent Bookcase · CF 813E Army Creation
- CF 484E Sign on Fence
- CSES 2072 · 2073 · 2074 — treap trilogy
- CF 863D · CF 702F T-Shirts (hard)

Summary

- Persistence = path copying: O(log n) nodes per update, all versions live
- Versions **subtract** → range × value queries in O(log n)
- Merge-sort tree: simplest range × value counter, $O(\log^2 n)$
- Offline? Sort + Fenwick first
- Treap: random priorities ⇒ expected O(log n), split/merge for everything
- Implicit treap = array with cut, paste, reverse
- Next: **Lecture 3 — DP optimizations** (Li Chao tree = segment tree of lines)